

TP : La classe CTFrameRtu

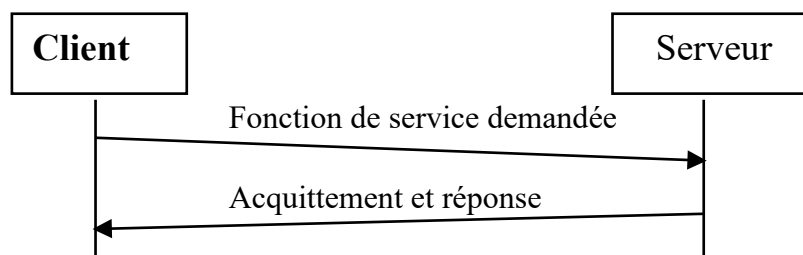
Ce TP a pour but de vous faire développer une classe CTFrameRtu capable de gérer les contrôles d'intégrité des trames RTU. Ce développement s'accompagnera du développement d'une classe CTest devant faciliter la mise en œuvre des tests de la classe CTFrameRtu.

Outils de développement PC sous Windows et/ou Linux : IDE au choix

Présentation du cahier des charges

Mise en situation :

Dans notre étude, votre PC est considéré comme un client chargé piloter un système distant serveur.



Le **Client** demande un service de contrôle au système **Serveur** distant par l'envoi d'une trame RTU que nous nommerons : "**trameService**".

Le **Serveur** répond pour s'acquitter du service demandé par l'envoi d'une trame RTU que nous nommerons : "**trameReponse**".

Spécifications :

Une trame RTU (Remote Terminal Unit) est une suite d'octets servant à transmettre des informations en mode dit **binaire** pour télécommander des systèmes distants.

Une trame RTU de N octets comprend 2 parties :

- les N-2 premiers octets servent à coder les informations transmises,
- les 2 derniers octets forment une clé CRC qui sert à vérifier et garantir l'intégrité des informations transmises.

Des exemples de trames RTU sont fournis dans l'annexe A.

Cette clé est le résultat d'un calcul effectué chez l'émetteur et vérifié chez le destinataire.

Un algorithme de calcul de cette clé CRC est fourni dans l'annexe B.

Besoin à satisfaire :

Il faut développer une classe **CTFrameRTU** capable gérer les contrôles d'intégrité des trames RTU au niveau du Client.

On peut séparer cette gestion en 2 cas d'utilisation :

- préparation de trame "**trameService**"
- contrôle de trame "**trameReponse**"

La préparation de trame "**trameService**" consiste à ajouter à la suite du code des informations à émettre la clé CRC de ce code.

Nous nommerons "**codeInformationEmise**" le code de l'information à émettre et "**crcEmis**" sa clé.

Le contrôle de trame "**trameReponse**" consiste à vérifier l'intégrité cette trame et d'en extraire le cas échéant le code de l'information reçue.

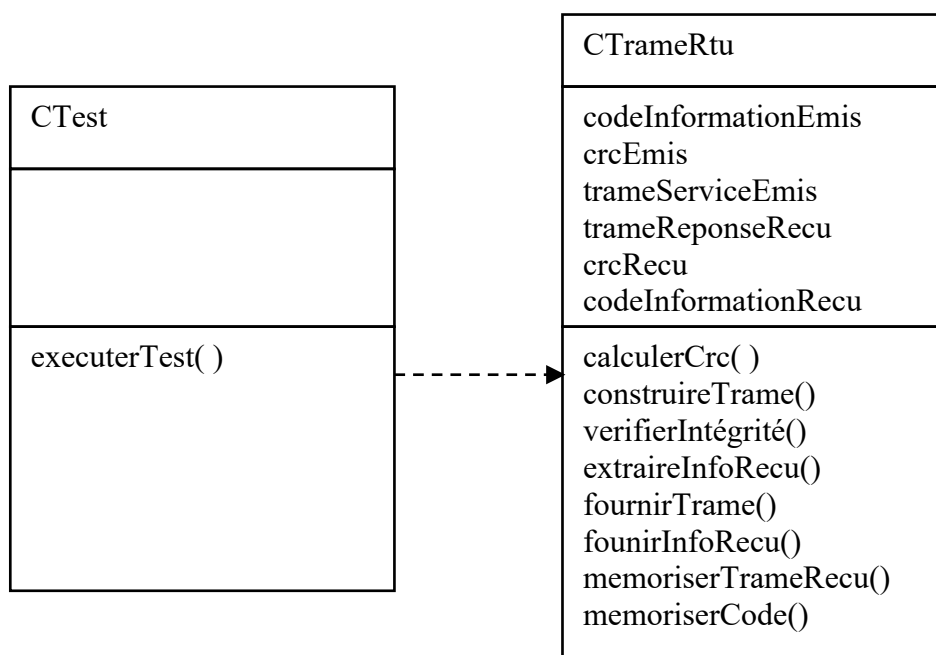
Nous nommerons "**codeInformationRecu**" du code de l'information à recevoir et "**crcRecu**" sa clé.

Nous ne nous intéressons pas dans ce développement au codage/décodage des informations transmises. Nous travaillons uniquement sur les codes bruts de ces informations. Le format des codes d'information utilisés varie de 2 octets à 12 octets.

Spécification de développement :

Le développement d'une classe CTest sera réalisé parallèlement au développement de la classe CTFrameRtu. L'application fonctionnera en mode console.

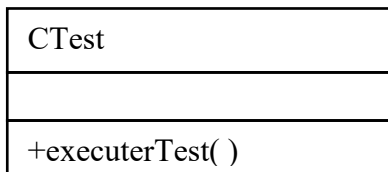
Le diagramme de classe suivant propose une implémentation de ces classes :



Travail à réaliser

Exercice 1 : Préparation de l'architecture de l'application en mode console.

Cet exercice a pour but de mettre en place l'instanciation de la Classe CTest simplifiée.



Description de la méthode à développer:

CTest::executerTest() : exécute la procédure de test limitée ici à afficher à l'écran un message : "Mon test s'exécute."

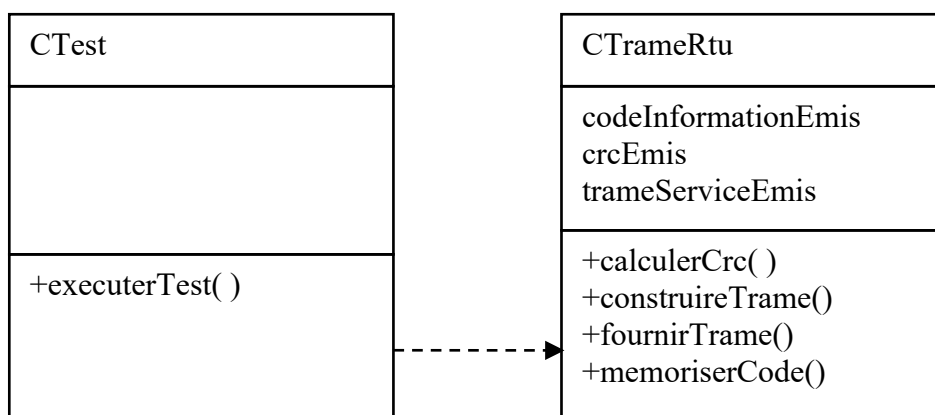
Le programme source principal qui sera nommé **monMain1.cpp** devra uniquement instancier un objet **monTest** de la Classe **CTest** et lancer l'exécution la méthode **ExecuterTest()**.
Après l'objet **monTest** devra être supprimé avant de terminer l'application.

La classe aura son code source dans un fichier **CTest1.cpp** indépendant et sera déclarée dans un fichier d'entête **CTest1.h** indépendant.

- 1) Créer un nouveau projet **projetTrameRtu1**.
- 2) Réalisez les programmes demandés dans **main.cpp**, **CTest1.cpp** et **CTest1.h**.
- 3) Sauvegardez, compilez et testez.
- 4) Mettez au point les sources et placez vos commentaires.
- 5) Prendre des notes et faire des relevés d'exécution en vue de la rédaction du compte rendu.

Exercice 2 : Développement N°2 de CTest et de CTFrameRtu

Cet exercice a pour but de mettre en place l'instanciation d'un objet maTrame de la Classe CTFrameRtu par une méthode de CTest et le développement d'une partie des méthodes de la classe CTFrameRtu conformément au diagramme de classe suivant.



Description des fonctions membres à développer :

Pour faciliter le test des fonctions membres de CTrameRtu, elles auront toute pour l'instant un statut public.

CTest::executerTest() : exécute l'algorithme de test que vous aurez défini.

CTrameRtu::fournirTrame() : exécute la procédure de fabrication de la trame à partir de **codeInformationEmis** en lançant successivement **calculerCrc()** , **construireTrame()** et mémorise la trame construite dans **trameServiceEmis**

CTrameRtu::memoriseCode() : mémorise le code fourni en paramètre dans **codeInformationEmis**

CTrameRtu::calculerCrc() : calcule **crcEmis** à partir de **codeInformationEmis**

CTrameRtu::construireTrame() : construit **trameService** à partir de **codeInformationEmis** et de **crcEmis**.

- 1) Choisir le type de chaque membre (attribut ou méthode) de CTrameRtu et justifier ces choix.
- 2) Préparer l'algorithme d'une procédure de test permettant la validation des développements réalisés en ayant pris soin de bien choisir le type chaque variable utilisée et le type de chaque méthode utilisée.
- 3) A partir du travail réalisé à l'exercice 1, créez un nouveau projet **projetTrameRtu2** avec tous les fichiers nommés respectivement : **main.cpp**, **CTest2.cpp**, **CTrameRtu2.cpp**, **CTest2.h** et **CTrameRtu2.h**.
- 4) Développez chaque méthode sans oublier de mettre à jour le fichier d'entête le fichier d'entête où doit être déclaré chaque classe.

Il est fortement conseillé :

- de développer les méthodes une à une si possible et de les tester progressivement en faisant évoluer la procédure de test au fur et à mesure de l'avancement du développement,
- de placer les commentaires au fur et à mesure de l'avancement du développement,
- de prendre des notes et de faire des relevés d'exécution en vue de la rédaction du compte rendu au fur et à mesure de l'avancement du développement.

Exercice 3 : Développement N°2 de CTest et de CTFrameRtu

Cet exercice a pour but de mettre en place l'instanciation d'un objet maTFrame de la Classe CTFrameRtu par une méthode de CTest et le développement des méthodes manquantes de la classe CTFrameRtu conformément au diagramme de classe de la page 2.

Description des fonctions membres restant à développer :

Pour faciliter le test des fonctions membres de CTFrameRtu, elles auront toutes pour l'instant un statut public.

CTest::executerTest() : exécute l'algorithme de test que vous aurez défini.

CTFrameRtu:: memoriseTFrameRecu () : mémorise la code fourni en paramètre dans **trameReponseRecu**.

CTFrameRtu:: extraireInfoRecu () : extrait de **trameReponseRecu** les valeurs de **crcRecu** et de **codeInformationRecu**.

CTFrameRtu::fournirInfoRecu() : retourne **codeInformationRecu** si **crcRecu** correct par rapport au **codeInformationRecu** sinon retourne un code constitué uniquement d'octets à 0.

- 1) Choisir le type de chaque membre ajouté(attribut ou méthode) de CTFrameRtu et justifier ces choix.
- 2) Préparer l'algorithme d'une procédure de test permettant la validation des développements réalisés en ayant pris soin de bien choisir le type chaque variable utilisée et de chaque méthode utilisée.
- 3) A partir du travail réalisé à l'exercice 2, créez un nouveau projet **projetFrameRtu3** avec tous les fichiers nommés respectivement : **main.cpp**, **CTest3.cpp**, **CTFrameRtu3.cpp**, **CTest3.h** et **CTFrameRtu3.h**.
- 4) Développez chaque méthode sans oublier de mettre à jour le fichier d'entête où doit être déclaré chaque classe.

Exercice 4 : Développement N°3 de CTest

Cet exercice a pour but de proposer une classe CTest plus évoluée qui propose un menu de tests avec des choix de paramétrage.

- 1) Proposer un menu de tests.
- 2) Inventorier et "typer" les membres et fonction membres nécessaires pour définir la nouvelle classe CTest. A partir du travail réalisé à l'exercice 3, créez un nouveau projet **projetFrameRtu4** avec tous les fichiers nommés respectivement : **main.cpp**, **CTest4.cpp**, **CTFrameRtu4.cpp**, **CTest4.h** et **CTFrameRtu4.h**.
- 3) Développez chaque méthode sans oublier de mettre à jour le fichier d'entête où doit être déclaré chaque classe.
- 4) Revoyez le statut de chaque membre et fonction membre de CTFrameRtu en le "privatisant" si possible.

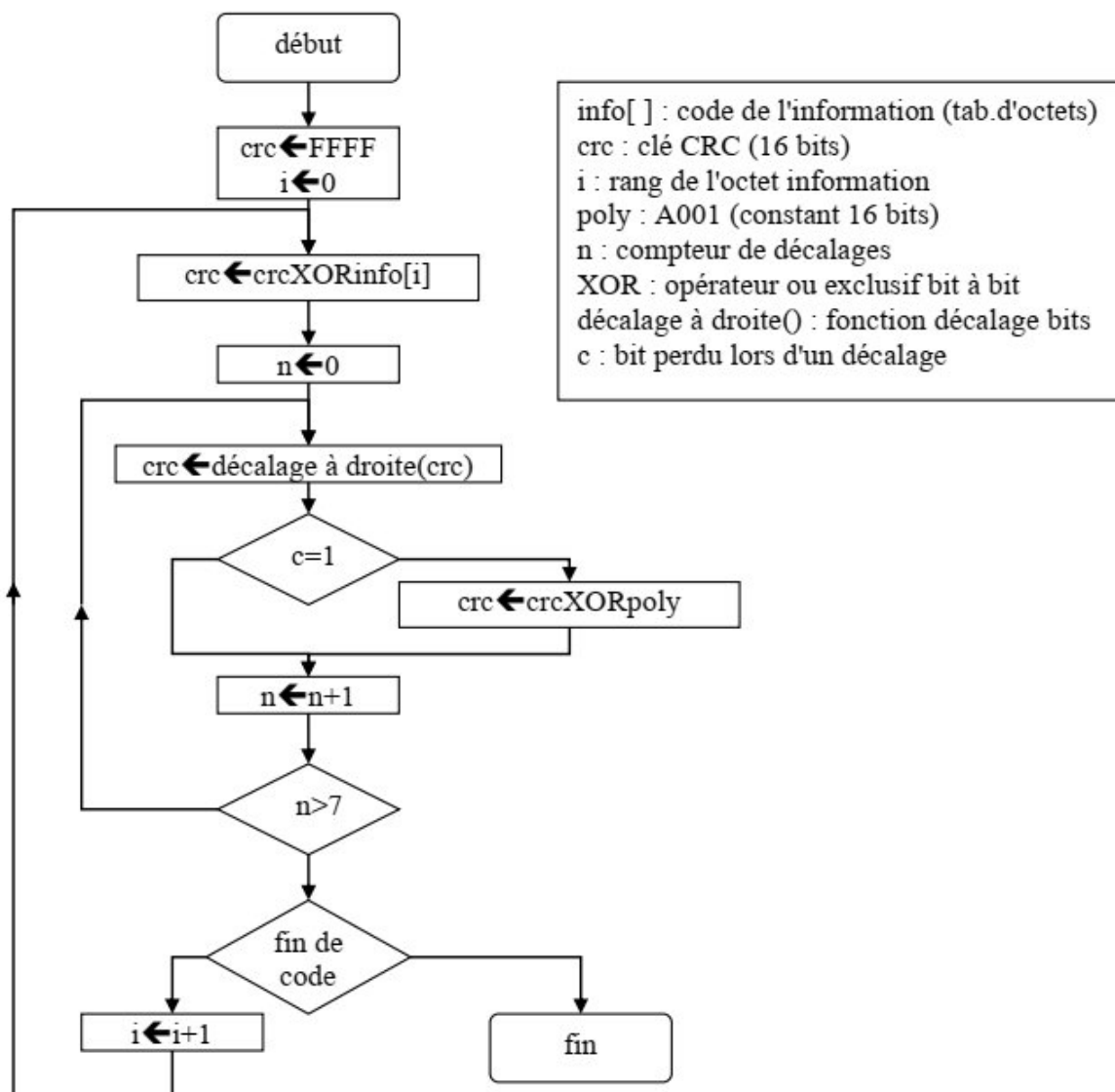
Annexe A : Exemples de frame RTU

Trame RTU							
Code de l'information transmise						Clé CRC	
0x09	0x03	0x00	0x22	0x00	0x02	0x65	0x49

Trame RTU												
Code de l'information transmise											Clé CRC	
0x09	0x10	0x00	0x51	0x00	0x02	0x04	0x00	0x56	0x00	0x57	0xBC	0xD1

Trame RTU			
Code de l'information transmise		Clé CRC	
0x02	0x07	0x41	0x12

Annexe B : Algorithme de calcul de la clé CRC



Attention pour la transmission : l'octet de poids faible de la clé CRC est transmis avant son octet de poids fort.